

[NAME DER UNIVERSITÄT]

[NAME DER FAKULTÄT]

[NAME DES LEHRSTUHLs/INSTITUTS]

Bachelorarbeit

Erweiterung des Tensor Power Flow um PV-Knoten: Methoden, Implementierung und Vergleich mit konventionellen Lastflussverfahren

Verfasser: [Vorname Nachname]

Matrikelnummer: [Matrikelnummer]

Studiengang: [Studiengang]

Erstprüfer: [Prof. Dr.-Ing. Name]

Zweitprüfer: [Prof. Dr.-Ing. Name]

Betreuer: [M.Sc. Name]

Beginn: [Datum]

Abgabe: [Datum]

[Stadt], [Monat] [Jahr]

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Bachelorarbeit selbstständig und ohne unzulässige Hilfe Dritter verfasst habe. Alle verwendeten Quellen und Hilfsmittel sind angegeben.

Ort, Datum

Unterschrift

Kurzfassung

Abstract

Inhaltsverzeichnis

Eidesstattliche Erklärung	i
Kurzfassung	iii
Abstract	v
Abbildungsverzeichnis	ix
Tabellenverzeichnis	xi
Symbolverzeichnis	xiii
Abkürzungsverzeichnis	xv
1 Grundlagen der Lastflussberechnung	1
1.1 Netzmodell und Knotentypen	1
1.1.1 Die Admittanzmatrix	1
1.1.2 Komplexe Leistung und Spannungsdarstellung	2
1.1.3 Knotentypen	2
1.1.4 Das ZIP-Lastmodell	3
1.2 Newton-Raphson-Verfahren	4
1.2.1 Die polare Standard-Formulierung	4
1.2.2 Unbekannte und Gleichungen	4
1.2.3 Iterationsvorschrift	5
1.2.4 Die Jacobi-Matrix in Blockform	5
1.2.5 Elemente der Jacobi-Matrix	6
1.2.6 Das vollständige Newton-Raphson-System	7
1.2.7 Behandlung von PV-Knoten	7
1.2.8 Q-Limits und PV-zu-PQ-Umwandlung	8
1.2.9 Konvergenzeigenschaften	8
1.2.10 Grenzen des Newton-Raphson-Verfahrens	9
1.3 Current Injection Method und PV-Behandlung	9
1.3.1 Strommismatch-Formulierung	10

1.3.2	Jacobi-Matrix der CIM	11
1.3.3	Behandlung von PV-Knoten in der CIM	13
1.3.4	Q-Limits und PV-zu-PQ-Umwandlung	15
1.4	Fixed-Point-Iteration und Konvergenz	15
1.4.1	Laurent-Reihen-Approximation	16
1.4.2	Herleitung des FPI-Algorithmus	16
1.4.3	Die Successive Approximation Method (SAM)	17
1.4.4	Konvergenzbeweis mittels Banach-Fixpunktsatz	18
1.4.5	Physikalische Interpretation der Konvergenzbedingung	19
2	Der Tensor Power Flow	21
2.1	Grundalgorithmus	21
2.2	Dense- und Sparse-Formulierung	22
2.2.1	Motivation: Der Tensor der Lastdaten	23
2.2.2	Dense-Formulierung	23
2.2.3	Sparse-Formulierung	24
2.3	Limitation: Keine PV-Knoten	25
2.3.1	Warum PV-Knoten diese Annahme verletzen	26
	Literaturverzeichnis	29
A	Ergänzende Herleitungen	31
A.1	Vollständige Herleitung der Sensitivitätskette	31
A.2	Koeffizienten der quadratischen Gleichung	31
B	Zusätzliche Simulationsergebnisse	33

Abbildungsverzeichnis

Tabellenverzeichnis

1.1	Klassifizierung der Knotentypen im Lastflussproblem.	3
1.2	Blöcke der Jacobi-Matrix in polarer Formulierung.	6

Symbolverzeichnis

Abkürzungsverzeichnis

Kapitel 1

Grundlagen der Lastflussberechnung

Die Lastflussberechnung (engl. *Power Flow*, PF) ist eine der fundamentalsten Analysen in der elektrischen Energietechnik. Sie dient der Bestimmung der stationären Spannungen und Ströme an allen Knoten eines Netzes unter gegebenen Erzeugungs- und Verbrauchsbedingungen. Dieses Kapitel führt die mathematischen Grundlagen ein, auf denen die in Kapitel 2 und ?? vorgestellten Methoden aufbauen.

1.1 Netzmodell und Knotentypen

1.1.1 Die Admittanzmatrix

Betrachtet wird ein Netz mit einem Einspeiseknoten (Umspannwerk), $b = |\Omega_d|$ Lastknoten und $\phi = |\Omega_\phi|$ Phasen. Die Beziehung zwischen Knotenspannungen und -strömen wird durch die Admittanzmatrix $\mathbf{Y} \in \mathbb{C}^{(b+1)\phi \times (b+1)\phi}$ beschrieben [1]:

$$\begin{bmatrix} \mathbf{i}_s \\ -\mathbf{i}_d \end{bmatrix} = \begin{bmatrix} \mathbf{Y}_{ss} & \mathbf{Y}_{sd} \\ \mathbf{Y}_{ds} & \mathbf{Y}_{dd} \end{bmatrix} \begin{bmatrix} \mathbf{v}_s \\ \mathbf{v}_d \end{bmatrix} \quad (1.1)$$

Dabei bezeichnen:

- $\mathbf{i}_s \in \mathbb{C}^{\phi \times 1}$: Vektor der komplexen Strominjektionen am Einspeiseknoten,
- $\mathbf{i}_d \in \mathbb{C}^{b\phi \times 1}$: Vektor der komplexen Strominjektionen an den Lastknoten $d \in \Omega_d$,
- $\mathbf{v}_s \in \mathbb{C}^{\phi \times 1}$: Vektor der komplexen Knotenspannungen am Einspeiseknoten,
- $\mathbf{v} := \mathbf{v}_d \in \mathbb{C}^{b\phi \times 1}$: Vektor der komplexen Knotenspannungen an den Lastknoten.

Die Admittanzmatrix ist in vier Blöcke unterteilt: $\mathbf{Y}_{ss}$ beschreibt die Selbstadmittanz des Einspeiseknotens, $\mathbf{Y}_{dd}$ die Verbindungen zwischen den Lastknoten untereinander,

und $\mathbf{Y}_{sd} = \mathbf{Y}_{ds}^T$ die Kopplung zwischen Einspeise- und Lastknoten. Die Formulierung in (1.1) ist allgemein und kann einphasige, mehrphasige, radiale sowie vermaschte Netze mit verteilter Erzeugung abbilden [1].

Die Admittanzmatrix wird aus den Leitungsimpedanzen des Netzes aufgebaut. Für eine Leitung zwischen Knoten i und j mit der Impedanz $z_{ij} = r_{ij} + jx_{ij}$ ergibt sich die Admittanz als $y_{ij} = 1/z_{ij}$. Die Matrixelemente sind:

$$Y_{ii} = \sum_{j \in \mathcal{N}_i} y_{ij} + y_{i,\text{shunt}} \quad (\text{Diagonalelemente}) \quad (1.2)$$

$$Y_{ij} = -y_{ij} \quad (\text{Außerdiagonalelemente}) \quad (1.3)$$

wobei $\mathcal{N}_i$ die Menge aller direkt mit Knoten i verbundenen Knoten bezeichnet und $y_{i,\text{shunt}}$ eine eventuelle Shunt-Admittanz am Knoten i ist.

1.1.2 Komplexe Leistung und Spannungsdarstellung

Die komplexe Scheinleistung an einem Knoten k ist definiert als:

$$s_k = P_k + jQ_k = v_k \cdot i_k^* \quad (1.4)$$

wobei P_k die Wirkleistung, Q_k die Blindleistung, v_k die komplexe Spannung und i_k^* der konjugiert komplexe Strom ist. Daraus folgt der Strom:

$$i_k = \left(\frac{s_k}{v_k} \right)^* = \frac{s_k^*}{v_k^*} = \frac{P_k - jQ_k}{v_k^*} \quad (1.5)$$

Die komplexe Spannung kann in zwei äquivalenten Formen dargestellt werden:

$$\text{Polarform: } V_k = |V_k| \cdot e^{j\theta_k} = |V_k| \angle \theta_k \quad (1.6)$$

$$\text{Kartesische Form: } V_k = e_k + jf_k \quad (1.7)$$

mit den Zusammenhängen $e_k = |V_k| \cos(\theta_k)$, $f_k = |V_k| \sin(\theta_k)$ und $|V_k| = \sqrt{e_k^2 + f_k^2}$. Beide Darstellungen finden in den nachfolgenden Verfahren Anwendung: Die Polarform wird klassisch im Newton-Raphson-Verfahren verwendet, die kartesische Form in der Current Injection Method und im Tensor Power Flow.

1.1.3 Knotentypen

Im Lastflussproblem werden die Netzknoten anhand der bekannten und gesuchten Größen in drei Typen eingeteilt. Tabelle 1.1 gibt eine Übersicht.

Tabelle 1.1: Klassifizierung der Knotentypen im Lastflussproblem.

Typ	Bekannt	Gesucht	Typische Anwendung
Slack ($V\delta$)	$ V , \theta$	P, Q	Umspannwerk, Referenzknoten
PQ	P, Q	$ V , \theta$	Verbraucher, PV-Wechselrichter mit $\cos \varphi$ -Regelung
PV	$P, V $	Q, θ	Synchrongeneratoren, Einspeiser mit Spannungsregelung

Slack-Knoten ($V\delta$ -Knoten). Der Slack-Knoten dient als Referenz für Spannung und Winkel im Netz. Sein Spannungsbetrag $|V_s|$ und sein Winkel θ_s sind vorgegeben. Die zugehörige Wirk- und Blindleistung ergibt sich als Resultat der Lastflussberechnung und gleicht die Leistungsbilanz des Netzes aus. Im Verteilnetz entspricht der Slack-Knoten typischerweise dem Umspannwerk.

PQ-Knoten. An PQ-Knoten sind die Wirkleistung P_k^{spec} und die Blindleistung Q_k^{spec} vorgegeben. Der Spannungsbetrag $|V_k|$ und der Winkel θ_k (bzw. e_k und f_k) sind die gesuchten Größen. Die meisten Knoten in einem Verteilnetz – Verbraucher, Wechselrichter mit festem Leistungsfaktor – werden als PQ-Knoten modelliert.

PV-Knoten. An PV-Knoten sind die Wirkleistung P_k^{spec} und der Spannungsbetrag $|V_k^{\text{spec}}|$ vorgegeben. Die Blindleistung Q_k ist unbekannt und muss so bestimmt werden, dass die Spannungsvorgabe eingehalten wird. PV-Knoten stellen eine besondere Herausforderung dar, da sie eine zusätzliche Nebenbedingung in das Gleichungssystem einführen:

$$|V_k|^2 = e_k^2 + f_k^2 = |V_k^{\text{spec}}|^2 \quad (1.8)$$

Aus (1.5) wird deutlich, warum PV-Knoten problematisch sind: Der Strom $i_k = (P_k - jQ_k)/v_k^*$ kann nicht direkt berechnet werden, da Q_k unbekannt ist. Im Gegensatz dazu ist bei PQ-Knoten die vollständige Leistung $s_k = P_k + jQ_k$ bekannt, sodass der Strom unmittelbar berechenbar ist.

1.1.4 Das ZIP-Lastmodell

In der Praxis hängt die aufgenommene Leistung einer Last vom Spannungsbetrag ab. Dies wird durch das ZIP-Modell berücksichtigt, das die Last als Kombination dreier Typen darstellt [2]:

$$\mathbf{S}_d = \left(\text{diag}(\alpha_P) + \text{diag}(\alpha_I |\mathbf{V}_d|) + \text{diag}(\alpha_Z |\mathbf{V}_d|^2) \right) \mathbf{S}_n \quad (1.9)$$

wobei $\mathbf{S}_n$ die nominale Leistung, und α_Z , α_I , α_P die Koeffizienten für die impedanz-, strom- und leistungskonstanten Anteile sind, mit $\alpha_Z + \alpha_I + \alpha_P = 1$. Der leistungskonstante Anteil (α_P) ist für die Nichtlinearität des Lastflussproblems verantwortlich.

1.2 Newton-Raphson-Verfahren

Das Newton-Raphson-Verfahren (NR) ist die am weitesten verbreitete Methode zur Lösung des Lastflussproblems in Übertragungs- und Verteilnetzen [3]. Es basiert auf einer Linearisierung der nichtlinearen Lastflussgleichungen mittels der Taylor-Reihe erster Ordnung und zeichnet sich durch quadratische Konvergenz aus.

1.2.1 Die polare Standard-Formulierung

In der klassischen polaren Darstellung werden die Leistungsgleichungen als Funktion der Spannungsbeträge $|V_k|$ und Spannungswinkel θ_k formuliert. Die komplexe Spannung an Knoten k ist dabei:

$$V_k = |V_k| \cdot e^{j\theta_k} \quad (1.10)$$

Aus der allgemeinen Leistungsbeziehung $S_k = V_k I_k^*$ und dem Zusammenhang $I_k = \sum_{m=1}^{n_b} Y_{km} V_m$ ergeben sich die Wirk- und Blindleistungsinjektionen an Knoten k zu [3]:

$$P_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (1.11)$$

$$Q_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (1.12)$$

wobei $G_{km} = \text{Re}(Y_{km})$ die Konduktanz und $B_{km} = \text{Im}(Y_{km})$ die Suszeptanz des Admittanzmatrix-Elements Y_{km} bezeichnen, und $\theta_k - \theta_m$ die Winkeldifferenz zwischen den Knoten k und m ist.

1.2.2 Unbekannte und Gleichungen

Die Struktur des Gleichungssystems hängt von den Knotentypen ab. Der Unbekanntenvektor $\Delta \mathbf{x}$ und der Mismatch-Vektor $\mathbf{g}(\mathbf{x})$ in der polaren Formulierung sind:

Unbekannte:

$$\Delta \mathbf{x} = \begin{bmatrix} \Delta \boldsymbol{\theta} \\ \Delta |\mathbf{V}| \end{bmatrix} \quad (1.13)$$

wobei $\Delta\boldsymbol{\theta} \in \mathbb{R}^{(n_b-1) \times 1}$ die Winkelkorrekturen an allen PQ- und PV-Knoten sind und $\Delta|\mathbf{V}| \in \mathbb{R}^{n_{PQ} \times 1}$ die Spannungsbetragskorrekturen **nur** an PQ-Knoten darstellen.

Mismatch-Gleichungen:

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} \Delta\mathbf{P} \\ \Delta\mathbf{Q} \end{bmatrix} = \mathbf{0} \quad (1.14)$$

wobei $\Delta\mathbf{P} \in \mathbb{R}^{(n_b-1) \times 1}$ die Wirkleistungsmismatches an allen PQ- und PV-Knoten und $\Delta\mathbf{Q} \in \mathbb{R}^{n_{PQ} \times 1}$ die Blindleistungsmismatches **nur** an PQ-Knoten sind. Die Mismatches sind definiert als:

$$\Delta P_k = P_k^{\text{spec}} - P_k(\boldsymbol{\theta}, |\mathbf{V}|) \quad \text{für } k \in \Omega_{PQ} \cup \Omega_{PV} \quad (1.15)$$

$$\Delta Q_k = Q_k^{\text{spec}} - Q_k(\boldsymbol{\theta}, |\mathbf{V}|) \quad \text{für } k \in \Omega_{PQ} \quad (1.16)$$

1.2.3 Iterationsvorschrift

Das NR-Verfahren linearisiert den Mismatch-Vektor $\mathbf{g}(\mathbf{x})$ um den aktuellen Arbeitspunkt $\mathbf{x}^{(k)}$ mittels einer Taylor-Entwicklung erster Ordnung:

$$\mathbf{g}(\mathbf{x}^{(k)} + \Delta\mathbf{x}) \approx \mathbf{g}(\mathbf{x}^{(k)}) + \mathbf{J}(\mathbf{x}^{(k)}) \cdot \Delta\mathbf{x} = \mathbf{0} \quad (1.17)$$

Daraus ergibt sich der Korrekturvektor:

$$\Delta\mathbf{x}^{(k)} = -\mathbf{J}(\mathbf{x}^{(k)})^{-1} \cdot \mathbf{g}(\mathbf{x}^{(k)}) \quad (1.18)$$

Die Zustandsgrößen werden iterativ aktualisiert gemäß:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)} \quad (1.19)$$

Der Algorithmus wird initialisiert mit dem *Flat Start*: $|V_k|^{(0)} = 1,0$ p.u. und $\theta_k^{(0)} = 0$ für alle Knoten (außer Slack). Die Iteration wird beendet, wenn $\max(|\Delta P_k|, |\Delta Q_k|) < \varepsilon$ mit typischen Toleranzen $\varepsilon = 10^{-6}$ bis 10^{-8} .

1.2.4 Die Jacobi-Matrix in Blockform

Die Jacobi-Matrix $\mathbf{J}$ enthält die partiellen Ableitungen der Mismatch-Funktionen nach den Unbekannten und wird in der polaren Formulierung in vier Blöcke unterteilt [3]:

$$\begin{bmatrix} \Delta\mathbf{P} \\ \Delta\mathbf{Q} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{H} & \mathbf{N} \\ \mathbf{M} & \mathbf{L} \end{bmatrix}}_{\mathbf{J}} \begin{bmatrix} \Delta\boldsymbol{\theta} \\ \Delta|\mathbf{V}| \end{bmatrix} \quad (1.20)$$

Die vier Untermatrizen haben folgende Bedeutung und Dimension:

Tabelle 1.2: Blöcke der Jacobi-Matrix in polarer Formulierung.

Block	Ableitung	Dimension	Physikalische Bedeutung
$\mathbf{H}$	$\frac{\partial \Delta P}{\partial \theta}$	$(n_b - 1) \times (n_b - 1)$	Einfluss der Winkel auf die Wirkleistung
$\mathbf{N}$	$\frac{\partial \Delta P}{\partial V }$	$(n_b - 1) \times n_{PQ}$	Einfluss der Beträge auf die Wirkleistung
$\mathbf{M}$	$\frac{\partial \Delta Q}{\partial \theta}$	$n_{PQ} \times (n_b - 1)$	Einfluss der Winkel auf die Blindleistung
$\mathbf{L}$	$\frac{\partial \Delta Q}{\partial V }$	$n_{PQ} \times n_{PQ}$	Einfluss der Beträge auf die Blindleistung

1.2.5 Elemente der Jacobi-Matrix

Die Elemente der vier Blöcke ergeben sich durch Ableitung der Leistungsgleichungen (1.11)–(1.12) nach θ_m und $|V_m|$. Es wird zwischen Diagonal- und Außerdiagonalelementen unterschieden.

Außerdiagonalelemente ($k \neq m$):

$$H_{km} = \frac{\partial P_k}{\partial \theta_m} = |V_k| |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (1.21)$$

$$N_{km} = \frac{\partial P_k}{\partial |V_m|} = |V_k| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (1.22)$$

$$M_{km} = \frac{\partial Q_k}{\partial \theta_m} = -|V_k| |V_m| [G_{km} \cos(\theta_k - \theta_m) + B_{km} \sin(\theta_k - \theta_m)] \quad (1.23)$$

$$L_{km} = \frac{\partial Q_k}{\partial |V_m|} = |V_k| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (1.24)$$

Diagonalelemente ($k = m$):

$$H_{kk} = \frac{\partial P_k}{\partial \theta_k} = -Q_k - B_{kk} |V_k|^2 \quad N_{kk} = \frac{\partial P_k}{\partial |V_k|} = \frac{P_k}{|V_k|} + G_{kk} |V_k| \quad (1.25)$$

$$M_{kk} = \frac{\partial Q_k}{\partial \theta_k} = P_k - G_{kk} |V_k|^2 \quad L_{kk} = \frac{\partial Q_k}{\partial |V_k|} = \frac{Q_k}{|V_k|} - B_{kk} |V_k| \quad (1.26)$$

Herleitung der Diagonalelemente. Die kompakte Form der Diagonalelemente ergibt sich durch Einsetzen der Leistungsdefinitionen. Beispielhaft für H_{kk} :

$$H_{kk} = \frac{\partial P_k}{\partial \theta_k} = |V_k| \sum_{\substack{m=1 \\ m \neq k}}^{n_b} |V_m| [-G_{km} \sin(\theta_k - \theta_m) + B_{km} \cos(\theta_k - \theta_m)] \quad (1.27)$$

Aus (1.12) erkennt man, dass dieser Ausdruck mit $-Q_k - B_{kk}|V_k|^2$ identisch ist, da der Anteil $m = k$ in der Summe den Term $-B_{kk}|V_k|^2$ liefert. Somit folgt die kompakte Darstellung in (??).

Zusammenhang zwischen den Blöcken. Aus den Gleichungen (1.21)–(1.24) lässt sich erkennen, dass für die Außerdiagonalelemente gilt:

$$H_{km} = -M_{km}, \quad N_{km} = L_{km} \quad (k \neq m) \quad (1.28)$$

Diese Symmetrieeigenschaft wird in der *Fast Decoupled Power Flow*-Methode ausgenutzt, ist aber für die vorliegende Arbeit nicht weiter relevant.

1.2.6 Das vollständige Newton-Raphson-System

Das in jeder Iteration k zu lösende lineare Gleichungssystem lautet ausgeschrieben:

$$\begin{bmatrix} \mathbf{H}^{(k)} & \mathbf{N}^{(k)} \\ \mathbf{M}^{(k)} & \mathbf{L}^{(k)} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{\theta}^{(k)} \\ \Delta |\mathbf{V}|^{(k)} \end{bmatrix} = \begin{bmatrix} \Delta \mathbf{P}^{(k)} \\ \Delta \mathbf{Q}^{(k)} \end{bmatrix} \quad (1.29)$$

Dieses System wird typischerweise durch LU-Zerlegung der dünnbesetzten (sparse) Jacobi-Matrix gelöst. Die Dünnbesetztheit resultiert aus der Tatsache, dass H_{km} , N_{km} , M_{km} , L_{km} nur dann von null verschieden sind, wenn eine direkte Verbindung (Leitung) zwischen den Knoten k und m besteht, d. h. $Y_{km} \neq 0$.

1.2.7 Behandlung von PV-Knoten

In der polaren Formulierung werden PV-Knoten **implizit** durch die Dimensionsreduktion des Gleichungssystems behandelt:

1. Der Spannungsbetrag $|V_k|$ ist fest vorgegeben $\Rightarrow \Delta|V_k|$ ist *keine* Unbekannte $\Rightarrow$ die Spalte für $\Delta|V_k|$ entfällt in $\mathbf{N}$ und $\mathbf{L}$.
2. Die Blindleistung Q_k ist unbekannt $\Rightarrow \Delta Q_k$ kann nicht als Mismatch formuliert werden $\Rightarrow$ die Zeile für ΔQ_k entfällt in $\mathbf{M}$ und $\mathbf{L}$.

Das resultierende System hat weiterhin gleich viele Gleichungen wie Unbekannte:

$$\begin{array}{lcl} \text{Unbekannte:} & \underbrace{(n_b - 1)}_{\Delta \theta \text{ (PQ + PV)}} & + \underbrace{n_{PQ}}_{\Delta |V| \text{ (nur PQ)}} \\ \text{Gleichungen:} & \underbrace{(n_b - 1)}_{\Delta P \text{ (PQ + PV)}} & + \underbrace{n_{PQ}}_{\Delta Q \text{ (nur PQ)}} \end{array}$$

Die Blindleistung Q_k am PV-Knoten ergibt sich nach Konvergenz implizit aus (1.12):

$$Q_k = |V_k| \sum_{m=1}^{n_b} |V_m| [G_{km} \sin(\theta_k - \theta_m) - B_{km} \cos(\theta_k - \theta_m)] \quad (1.30)$$

1.2.8 Q-Limits und PV-zu-PQ-Umwandlung

Generatoren an PV-Knoten verfügen über begrenzte Blindleistungskapazität $Q_k \in [Q_k^{\min}, Q_k^{\max}]$. Nach jeder Iteration wird geprüft, ob die aus (1.30) berechnete Blindleistung innerhalb der Grenzen liegt:

$$Q_k^{\min} \leq Q_k \leq Q_k^{\max} \quad (1.31)$$

Falls ein Limit verletzt wird, erfolgt eine **PV-zu-PQ-Umwandlung**:

- Die Blindleistung wird auf das verletzte Limit fixiert: $Q_k = Q_k^{\min}$ oder $Q_k = Q_k^{\max}$.
- Der Knoten wird fortan als PQ-Knoten behandelt ($|V_k|$ wird frei, ΔQ_k wird als Gleichung hinzugefügt).
- Die Jacobi-Matrix erhält eine zusätzliche Zeile (ΔQ_k) und eine zusätzliche Spalte ($\Delta |V_k|$).

In nachfolgenden Iterationen wird geprüft, ob der Spannungsbetrag wieder innerhalb des vorgegebenen Werts liegt, und der Knoten gegebenenfalls zurück in den PV-Typ konvertiert wird.

1.2.9 Konvergenzeigenschaften

Das NR-Verfahren weist **quadratische Konvergenz** auf, d. h. der Fehler wird in jeder Iteration quadriert:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq c \cdot \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2 \quad (1.32)$$

wobei $\mathbf{x}^*$ die exakte Lösung und c eine Konstante ist. Dies führt typischerweise zu einer Konvergenz in 3–6 Iterationen, weitgehend unabhängig von der Netzgröße [3]. Ein wesentlicher Vorteil des NR-Verfahrens in polarer Form ist, dass die Kopplung zwischen allen Unbekannten ($\Delta\theta$, $\Delta|V|$) *simultan* im linearen Gleichungssystem (1.29) berücksichtigt wird. Dadurch wird die volle Wechselwirkung zwischen Wirk- und Blindleistung, Winkeln und Beträgen in jedem Schritt ausgenutzt.

1.2.10 Grenzen des Newton-Raphson-Verfahrens

Trotz seiner quadratischen Konvergenz weist das NR-Verfahren Nachteile auf, die bei der Berechnung großer Mengen von Lastflüssen relevant werden:

1. Die Jacobi-Matrix $\mathbf{J}(\mathbf{x}^{(k)})$ muss in *jeder* Iteration neu berechnet werden, da ihre Elemente (1.21)–(1.26) von den aktuellen Spannungswerten abhängen.
2. Die Konvergenz hängt vom Startwert ab; bei ungünstiger Initialisierung kann das Verfahren zur nicht-operationellen Lösung (niedrige Spannung, hoher Strom) konvergieren oder divergieren [1].
3. Eine Parallelisierung über mehrere Lastflüsse ist schwierig, da jeder Lastfluss seine eigene, iterationsabhängige Jacobi-Matrix besitzt.

Diese Einschränkungen motivieren die Suche nach alternativen Verfahren, insbesondere für Anwendungen, die Hunderttausende von Lastflüssen erfordern. Der in Kapitel 2 vorgestellte Tensor Power Flow adressiert genau diese Limitierungen, kann dafür aber keine PV-Knoten direkt abbilden.

1.3 Current Injection Method und PV-Behandlung

Die konventionelle Newton-Raphson-Lastflussberechnung basiert auf Leistungs-Mismatch-Gleichungen, die in Polarkoordinaten formuliert werden. Dabei müssen in jedem Iterationsschritt trigonometrische Funktionen (Sinus, Kosinus) ausgewertet und die Jacobi-Matrix vollständig neu berechnet werden, was insbesondere bei großen Netzen einen erheblichen Rechenaufwand darstellt. Die *Current Injection Method* (CIM), vorgestellt von da Costa et al. [4], reformuliert das Lastflussproblem auf Basis von Strommismatch-Gleichungen in kartesischen Koordinaten. Dieser Ansatz führt zu einer Jacobi-Matrix, deren Struktur identisch mit der $(2n \times 2n)$ -Knotenadmittanzmatrix ist und deren außerdiagonale Blöcke – mit Ausnahme der PV-Knoten – während des gesamten Iterationsprozesses konstant bleiben. Dadurch entfällt die Notwendigkeit, transzendente Funktionen auszuwerten, und es ergibt sich eine signifikante Reduktion der Rechenzeit von über 20 % gegenüber der konventionellen Formulierung [4].

Im Folgenden wird zunächst die Strommismatch-Formulierung für PQ-Knoten hergeleitet (Abschnitt 1.3.1), anschließend die Struktur der resultierenden Jacobi-Matrix erläutert (Abschnitt 1.3.2), die Erweiterung auf PV-Knoten beschrieben (Abschnitt 1.3.3) und schließlich die Behandlung von Blindleistungsgrenzen mit der PV-zu-PQ-Umwandlung dargestellt (Abschnitt 1.3.4).

1.3.1 Strommismatch-Formulierung

Ausgangspunkt der CIM ist die komplexe Strominjektion an einem Knoten k . Die Netzgleichung in Matrixform lautet:

$$\mathbf{I} = \mathbf{Y} \mathbf{V} \quad (1.33)$$

wobei $\mathbf{Y} \in \mathbb{C}^{n \times n}$ die Knotenadmittanzmatrix, $\mathbf{V} \in \mathbb{C}^n$ der Vektor der komplexen Knotenspannungen und $\mathbf{I} \in \mathbb{C}^n$ der Vektor der komplexen Strominjektionen ist.

Für einen PQ-Knoten k ist die geplante komplexe Leistungsinjektion $S_k^{\text{sp}} = P_k^{\text{sp}} + j Q_k^{\text{sp}}$ bekannt. Die geplante (scheduled) Strominjektion ergibt sich aus der Beziehung zwischen Leistung und Strom:

$$I_k^{\text{sp}} = \left(\frac{S_k^{\text{sp}}}{V_k} \right)^* = \frac{P_k^{\text{sp}} - j Q_k^{\text{sp}}}{V_k^*} \quad (1.34)$$

wobei $V_k^* = e_k - j f_k$ die konjugiert komplexe Spannung am Knoten k bezeichnet, mit dem Realteil e_k und dem Imaginärteil f_k .

Die berechnete Strominjektion am Knoten k ergibt sich aus der Knotenadmittanzmatrix:

$$I_k^{\text{calc}} = \sum_{m=1}^n Y_{km} V_m = \sum_{m=1}^n (G_{km} + j B_{km})(e_m + j f_m) \quad (1.35)$$

Der komplexe Strommismatch am Knoten k ist definiert als die Differenz zwischen geplanter und berechneter Strominjektion [4]:

$$\Delta I_k = I_k^{\text{sp}} - I_k^{\text{calc}} = \Delta \mathcal{A}_k + j \Delta \mathcal{M}_k \quad (1.36)$$

Durch Separation in Real- und Imaginärteil ergeben sich zwei reelle Gleichungen pro Knoten. Dabei lassen sich die Stromkomponenten über die Leistungs-Mismatches ΔP_k und ΔQ_k ausdrücken. Die Wirk- und Blindleistungs-Mismatches sind definiert als:

$$\Delta P_k = P_k^{\text{sp}} - P_k^{\text{calc}} \quad (1.37)$$

$$\Delta Q_k = Q_k^{\text{sp}} - Q_k^{\text{calc}} \quad (1.38)$$

wobei die berechneten Leistungen durch die Spannungen und den Stromvektor gegeben sind:

$$P_k^{\text{calc}} = e_k \mathcal{A}_k^{\text{calc}} + f_k \mathcal{M}_k^{\text{calc}} \quad (1.39)$$

$$Q_k^{\text{calc}} = f_k \mathcal{A}_k^{\text{calc}} - e_k \mathcal{M}_k^{\text{calc}} \quad (1.40)$$

Durch algebraische Umformung lassen sich die Strommismatch-Komponenten di-

rekt über die bekannten Leistungs-Mismatches und die aktuellen Spannungswerte berechnen [4]:

$$\Delta_{\mathcal{A}_k} = \frac{e_k \Delta P_k + f_k \Delta Q_k}{V_k^2} \quad (1.41)$$

$$\Delta_{\mathcal{M}_k} = \frac{f_k \Delta P_k - e_k \Delta Q_k}{V_k^2} \quad (1.42)$$

mit $V_k^2 = e_k^2 + f_k^2$. Diese Formulierung ist für PQ-Knoten direkt auswertbar, da sowohl ΔP_k als auch ΔQ_k bekannte Größen sind. Die Gleichungen (1.41) und (1.42) bilden das Gleichungssystem, welches im Newton-Raphson-Verfahren iterativ gelöst wird.

Die Lastmodellierung erfolgt gemäß dem in Abschnitt 1.1.4 eingeführten ZIP-Modell in polynomialer Form:

$$P_{L(k)} = P_{0k} (\alpha_Z V_k^2 + \alpha_I V_k + \alpha_P) \quad (1.43)$$

$$Q_{L(k)} = Q_{0k} (\alpha_Z V_k^2 + \alpha_I V_k + \alpha_P) \quad (1.44)$$

wobei die Koeffizienten die Bedingung $\alpha_Z + \alpha_I + \alpha_P = 1$ erfüllen müssen. Die Terme α_Z , α_I und α_P repräsentieren die Anteile der konstanten Impedanz-, konstanten Strom- und konstanten Leistungslast (vgl. Gleichung (1.9)).

1.3.2 Jacobi-Matrix der CIM

Das Newton-Raphson-Verfahren, angewandt auf die Strommismatch-Gleichungen, führt zu folgendem linearen Gleichungssystem pro Iterationsschritt:

$$\begin{bmatrix} \Delta_{\mathcal{M}_1} \\ \Delta_{\mathcal{A}_1} \\ \vdots \\ \Delta_{\mathcal{M}_n} \\ \Delta_{\mathcal{A}_n} \end{bmatrix} = \mathbf{J} \begin{bmatrix} \Delta e_1 \\ \Delta f_1 \\ \vdots \\ \Delta e_n \\ \Delta f_n \end{bmatrix} \quad (1.45)$$

Die Anordnung der Gleichungen erfolgt dabei so, dass die Imaginärteil-Gleichungen ($\Delta_{\mathcal{M}_k}$) vor den Realteil-Gleichungen ($\Delta_{\mathcal{A}_k}$) stehen. Diese Sortierung bewirkt, dass die Jacobi-Matrix $\mathbf{J}^*$ diagonaldominant wird, da die Suszeptanzen B_{km} betragsmäßig typischerweise deutlich größer als die Konduktanzen G_{km} sind [4].

Die Jacobi-Matrix $\mathbf{J}^*$ besitzt die gleiche Blockstruktur wie die $(2n \times 2n)$ -Knotenadmittanzmatrix, wobei jeder Netzwerkzweig durch einen (2×2) -Block repräsentiert wird. Für die

Außerdiagonalelemente gilt bei Verbindung zwischen PQ-Knoten k und m :

$$\mathbf{J}_{km} = \begin{bmatrix} -B_{km} & G_{km} \\ G_{km} & B_{km} \end{bmatrix} \quad (1.46)$$

Diese Blöcke sind identisch mit den entsprechenden Elementen der Knotenadmittanzmatrix in kartesischer Darstellung und bleiben daher während des gesamten Iterationsprozesses **konstant**. Dies ist der zentrale Vorteil der CIM gegenüber der konventionellen Formulierung: Die aufwendige Neuberechnung der Außerdiagonalelemente entfällt vollständig.

Die (2×2) -Diagonalblöcke für einen PQ-Knoten k haben die Form:

$$\mathbf{J}_{kk} = \begin{bmatrix} -B'_{kk} & G'_{kk} \\ G''_{kk} & B''_{kk} \end{bmatrix} \quad (1.47)$$

mit den modifizierten Elementen [4]:

$$B'_{kk} = B_{kk} - a_k \quad (1.48)$$

$$G'_{kk} = G_{kk} - b_k \quad (1.49)$$

$$G''_{kk} = G_{kk} - c_k \quad (1.50)$$

$$B''_{kk} = -B_{kk} - d_k \quad (1.51)$$

Die Terme a_k , b_k , c_k und d_k hängen von der spezifizierten Last und Erzeugung am Knoten k sowie vom gewählten Lastmodell ab. Für den Fall einer reinen Konstantleistungslast ($\alpha_P = 1$, $\alpha_Z = \alpha_I = 0$) vereinfachen sich diese zu [4]:

$$a_k = -d_k = \frac{Q'_k(e_k^2 - f_k^2) - 2e_k f_k P'_k}{V_k^4} \quad (1.52)$$

$$b_k = -c_k = \frac{P'_k e_k^2 - f_k^2 + 2e_k f_k Q'_k}{V_k^4} \quad (1.53)$$

mit

$$P'_k = P_{G(k)} - P_{0k} \alpha_P \quad (1.54)$$

$$Q'_k = Q_{G(k)} - Q_{0k} \alpha_P \quad (1.55)$$

wobei $P_{G(k)}$ und $Q_{G(k)}$ die erzeugte Wirk- bzw. Blindleistung am Knoten k bezeichnen und P_{0k} , Q_{0k} die Nennlastwerte sind.

Da diese Terme von der aktuellen Spannung V_k abhängen, müssen die Diagonalblöcke in jedem Iterationsschritt aktualisiert werden. Für Knoten ohne spezifizierte Last oder mit reiner Impedanzlast ($a_p = 1$, $b_p = c_p = 0$) reduzieren sich die Terme a_k , b_k ,

c_k und d_k auf konstante Admittanzanteile, sodass der Diagonalblock während des gesamten Iterationsprozesses unverändert bleibt [4].

Die Spannungskorrekturen werden nach der Lösung des linearen Gleichungssystems in kartesischen Koordinaten berechnet und können anschließend in polare Koordinaten umgerechnet werden:

$$\Delta\theta_k = \arctan\left(\frac{e_k \Delta f_k - f_k \Delta e_k}{e_k^2 + f_k^2}\right) \quad (1.56)$$

$$\Delta V_k = \frac{e_k \Delta e_k + f_k \Delta f_k}{V_k} \quad (1.57)$$

Die Aktualisierung der Zustandsgrößen erfolgt gemäß:

$$\theta_k^{(h+1)} = \theta_k^{(h)} + \Delta\theta_k^{(h+1)} \quad (1.58)$$

$$V_k^{(h+1)} = V_k^{(h)} + \Delta V_k^{(h+1)} \quad (1.59)$$

wobei h den Iterationszähler bezeichnet. Es ist hervorzuheben, dass die Konvergenztrajektorie der CIM identisch mit der des konventionellen Newton-Raphson-Verfahrens in Polarkoordinaten ist [4].

1.3.3 Behandlung von PV-Knoten in der CIM

An PV-Knoten (Generatorknoten) sind die Wirkleistungseinspeisung P_k^{sp} und der Spannungsbetrag V_k^{sp} vorgegeben, während die Blindleistung Q_k eine unbekannte (abhängige) Variable darstellt. Dies erfordert eine besondere Behandlung innerhalb der CIM, da der Blindleistungs-Mismatch ΔQ_k in den Gleichungen (1.41) und (1.42) nicht direkt bekannt ist.

Der in [4] vorgeschlagene Ansatz löst dieses Problem elegant durch die Einführung einer zusätzlichen abhängigen Variablen ΔQ_k für jeden PV-Knoten sowie einer zusätzlichen linearisierten Gleichung, die die Bedingung eines konstanten Spannungsbetrags $|\Delta V_k| = 0$ erzwingt. Die linearisierte Form dieser Bedingung lautet:

$$e_k \Delta e_k + f_k \Delta f_k = 0 \quad (1.60)$$

Diese Gleichung kann umgestellt werden zu:

$$\Delta f_k = -\frac{e_k}{f_k} \Delta e_k \quad (1.61)$$

Durch Einsetzen von Gleichung (1.61) in die Strommismatch-Gleichungen des Knotens k wird die abhängige Variable ΔQ_k (in Form von $\Delta Q_k/V_k^2$) anstelle von Δf_k als Unbekannte im Gleichungssystem verwendet. Dies bedeutet, dass im Lösungsvektor

die Komponente Δf_k für jeden PV-Knoten durch $\Delta Q_k/V_k^2$ ersetzt wird.

Das resultierende erweiterte Gleichungssystem für einen PV-Knoten k lässt sich in Blockform schreiben als:

$$\begin{bmatrix} \Delta M_k \\ \Delta \Pi_k \\ 0 \end{bmatrix} = \begin{bmatrix} -B_{kk} & G_{kk} & f_k/V_k^2 \\ G_{kk} & B_{kk} & -e_k/V_k^2 \\ e_k & f_k & 0 \end{bmatrix} \begin{bmatrix} \Delta e_k \\ \Delta f_k \\ \Delta Q_k \end{bmatrix} \quad (1.62)$$

Durch Elimination der dritten Zeile und Substitution von Δf_k ergibt sich ein (2×2) -Diagonalblock für den PV-Knoten k :

$$\mathbf{J}_{kk,\text{PV}} = \begin{bmatrix} -B_{kk} - G_{kk} \frac{e_k}{f_k} & \frac{f_k}{V_k^2} \\ G_{kk} + B_{kk} \frac{e_k}{f_k} & -\frac{e_k}{V_k^2} \end{bmatrix} \quad (1.63)$$

Die außerdiagonalen (2×2) -Blöcke, die einen PV-Knoten k mit einem benachbarten Knoten l verbinden, verändern sich ebenfalls. Die erste Spalte bleibt identisch mit der Admittanzmatrix, während die zweite Spalte zu Null wird, da Δf_k durch die Spannungsbedingung eliminiert wurde:

$$\mathbf{J}_{lk,\text{PV}} = \begin{bmatrix} -B_{lk} - G_{lk} \frac{e_k}{f_k} & 0 \\ G_{lk} + B_{lk} \frac{e_k}{f_k} & 0 \end{bmatrix} \quad (1.64)$$

Diese Implementierung bewahrt die $(2n \times 2n)$ -Struktur der Jacobi-Matrix unabhängig von der Anzahl der PV-Knoten im System. Die Unbekannte Δf_k wird durch ΔQ_k ersetzt, sodass nach der Lösung des Gleichungssystems der aktualisierte Blindleistungs-Mismatch direkt verfügbar ist. Die Spannung am PV-Knoten wird ausschließlich über die Winkelkorrektur aktualisiert:

$$\theta_k^{(h+1)} = \theta_k^{(h)} + \Delta \theta_k^{(h+1)} \quad (1.65)$$

während der Spannungsbetrag $V_k = V_k^{\text{sp}}$ konstant bleibt.

Abbildung ?? illustriert schematisch die Struktur der Jacobi-Matrix $\mathbf{J}^*$ für ein System mit PQ- und PV-Knoten. Die mit einem Stern (*) markierten Elemente werden in jeder Iteration aktualisiert, während alle übrigen Elemente konstant bleiben.

Ein wesentlicher Vorteil dieser Formulierung ist, dass die Konvergenzcharakteristik vollständig mit der des konventionellen Newton-Raphson-Verfahrens in Polarkoordinaten übereinstimmt. Dies wurde von da Costa et al. [4] anhand praxisrelevanter Testnetze mit bis zu 2111 Knoten nachgewiesen, wobei identische Konvergenzverläufe bei einer gleichzeitigen Reduktion der Rechenzeit um über 20% erzielt wurden.

Die Rechenzeiteinsparung resultiert primär aus der Konstanz der außerdiagonalen Jacobi-Elemente sowie dem Entfall trigonometrischer Funktionsauswertungen.

1.3.4 Q-Limits und PV-zu-PQ-Umwandlung

In realen Energieversorgungssystemen unterliegen Generatoren physikalischen Blindleistungsgrenzen:

$$Q_k^{\min} \leq Q_k \leq Q_k^{\max} \quad (1.66)$$

Solange die am PV-Knoten berechnete Blindleistung Q_k innerhalb dieser Grenzen liegt, kann der Generator die spezifizierte Spannung aufrechterhalten. Wird eine der Grenzen verletzt, muss der PV-Knoten in einen PQ-Knoten umgewandelt werden, wobei Q_k^{sp} auf den verletzten Grenzwert fixiert wird.

Bei der Umwandlung ändert sich die Struktur der Jacobi-Matrix lokal:

- Der (2×2) -Diagonalblock wechselt von der PV-Form (Gleichung (1.63)) zur PQ-Form (Gleichung (1.47)), wobei Q_k^{sp} den fixierten Grenzwert annimmt.
- Die außerdiagonalen Blöcke in der Spalte des betroffenen Knotens kehren zur Standardform gemäß Gleichung (1.46) zurück.
- Die Unbekannte ΔQ_k wird durch ΔV_{mk} ersetzt, sodass der Spannungsbetrag frei variieren kann.

Die Rückumwandlung in einen PV-Knoten erfolgt, wenn die berechnete Spannung den Sollwert V_k^{sp} überschreitet (bei Fixierung auf $Q_k^{\min}$) bzw. unterschreitet (bei Fixierung auf $Q_k^{\max}$). Um Oszillationen zwischen den Knotentypen zu vermeiden, empfiehlt es sich, die Umwandlungslogik erst nach einigen Anfangsiterationen zu aktivieren.

Dieser Wechsel ist innerhalb der CIM besonders effizient, da lediglich lokale Modifikationen an einzelnen (2×2) -Blöcken erforderlich sind und die $(2n \times 2n)$ -Gesamtstruktur sowie die Sparse-Faktorisierungsreihenfolge unverändert bleiben.

1.4 Fixed-Point-Iteration und Konvergenz

Die Fixed-Point-Iteration (FPI), auch als *Successive Approximation Method* (SAM) bezeichnet, bietet eine Alternative zum Newton-Raphson-Verfahren, die sich durch ihre einfache Formulierung, geringe Kosten pro Iteration und natürliche Eignung zur Parallelisierung auszeichnet. Die Methode wurde von Giraldo et al. [2] im Rahmen der Current Injection Method unter Verwendung der Laurent-Reihenentwicklung vorgeschlagen und bildet die Grundlage des in Kapitel 2 vorgestellten Tensor Power Flow.

1.4.1 Laurent-Reihen-Approximation

Die Nichtlinearität der Lastflussgleichungen (??) resultiert aus dem Term $(V_k^\phi)^{-1}$, der im spezifizierten Strom (1.5) auftritt. Giraldo et al. [2] schlagen eine Linearisierung mittels Laurent-Reihenentwicklung vor.

Proposition 1.1 (Laurent-Approximation, [2, Proposition 1]). *Die Nichtlinearität $f(V_i) = (V_i)^{-1}$ an einem Knoten $i \in \Omega_d$ wird um einen Arbeitspunkt $V^0 \in \mathbb{C}$ mittels Laurent-Reihe approximiert als:*

$$f(V_i) \approx \frac{2}{V^0} - \frac{V_i}{(V^0)^2} \quad (1.67)$$

unter der Annahme $V_i = V^0 - \Delta V$ mit $\|\Delta V\|/\|V^0\| < 1$.

Die Annahme $\|\Delta V\|/\|V^0\| < 1$ ist physikalisch gerechtfertigt, da Knotenspannungen in Verteilnetzen im Normalbetrieb innerhalb eines engen Bandes um den Nennwert liegen (typisch 0,9 bis 1,1 p.u.) [2]. Im iterativen Algorithmus wird der Arbeitspunkt V^0 nach jeder Iteration auf den aktuellen Spannungswert aktualisiert, sodass die Approximation mit jeder Iteration genauer wird.

1.4.2 Herleitung des FPI-Algorithmus

Ausgangspunkt ist die Knotengleichung für die Lastknoten aus (1.1):

$$-\mathbf{I}_d = Y_{ds}\mathbf{V}_s + Y_{dd}\mathbf{V}_d \quad (1.68)$$

Der Strom an den Lastknoten ergibt sich unter Berücksichtigung des ZIP-Lastmodells (vgl. Abschnitt 1.1.4) nach Giraldo et al. [2] zu:

$$\mathbf{I}_d = \left(\alpha_P \text{diag}(\mathbf{V}_d^*)^{-1} + \text{diag}(\alpha_I) + \alpha_Z \text{diag}(\mathbf{V}_d) \right) \mathbf{S}_n^* \quad (1.69)$$

Durch Anwendung der Laurent-Approximation (Theorem 1.1) auf den nichtlinearen Term $\text{diag}(\mathbf{V}_d^*)^{-1}$ und Einsetzen in (1.68) ergibt sich das folgende lineare Gleichungssystem [2]:

$$\mathbf{A}\mathbf{V}_d^* - \mathbf{B}\mathbf{V}_d = \mathbf{C} + \mathbf{D} \quad (1.70)$$

mit den Matrizen und Vektoren:

$$\mathbf{A} = \text{diag}(\alpha_P \odot V^{0*-2} \odot \mathbf{S}_n^*) \quad (1.71)$$

$$\mathbf{B} = \text{diag}(\alpha_Z \odot \mathbf{S}_n^*) + Y_{dd} \quad (1.72)$$

$$\mathbf{C} = Y_{ds}\mathbf{V}_s + \alpha_I \odot \mathbf{S}_n^* \quad (1.73)$$

$$\mathbf{D} = 2\alpha_P \odot V^{0*-1} \odot \mathbf{S}_n^* \quad (1.74)$$

Hierbei bezeichnen $\odot$ das Hadamard-Produkt, V^{0*-1} den element-weisen Kehrwert des konjugierten Arbeitspunktvektors, und V^{0*-2} das element-weise Quadrat davon. Die Matrix $\mathbf{B} \in \mathbb{C}^{b\phi \times b\phi}$ und der Vektor $\mathbf{C} \in \mathbb{C}^{b\phi \times 1}$ sind innerhalb des iterativen Prozesses *konstant*, da sie nur von der Netztopologie (Y_{dd} , Y_{ds}), der Slack-Spannung ($\mathbf{V}_s$) und den spannungsunabhängigen Lastanteilen abhängen. Lediglich $\mathbf{A}$ und $\mathbf{D}$ müssen in jeder Iteration aktualisiert werden, da sie vom Arbeitspunkt V^0 abhängen [2].

1.4.3 Die Successive Approximation Method (SAM)

Giraldo et al. [2] schlagen vor, das Gleichungssystem (1.70) nach $\mathbf{V}_d$ umzustellen:

$$\mathbf{V}_d^{(k+1)} = \mathbf{B}^{-1} \left(\mathbf{A}^{(k)} \mathbf{V}_d^{*(k)} - \mathbf{C} - \mathbf{D}^{(k)} \right) \quad (1.75)$$

Diese Gleichung hat die Struktur einer Fixpunktiteration $\mathbf{V}_d^{(k+1)} = f(\mathbf{V}_d^{(k)})$ und kann sukzessiv gelöst werden. Der Algorithmus ist in Algorithmus 1 dargestellt.

Algorithm 1 Successive Approximation Method (SAM) nach [2]

Require: $\mathbf{S}_n$, Y_{dd} , Y_{ds} , $\mathbf{V}_s$, Toleranz ε , max. Iterationen K

Ensure: $\mathbf{V}_d$

- 1: Initialisiere $k = 0$, $\text{tol} = \infty$
 - 2: Initialisiere $V_{i,\phi}^0 = [1, a^2, a]$ für alle $i \in \Omega_d$ {Flat Start, $a = e^{j2\pi/3}$ }
 - 3: Berechne $\mathbf{B}^{-1}$ und $\mathbf{C}$ gemäß (1.72)–(1.73) {Einmalig!}
 - 4: **while** $\text{tol} \geq \varepsilon$ **und** $k < K$ **do**
 - 5: Berechne $\mathbf{A}^{(k)}$ und $\mathbf{D}^{(k)}$ gemäß (1.71)–(1.74)
 - 6: $\mathbf{V}_d^{(k+1)} = \mathbf{B}^{-1} \left(\mathbf{A}^{(k)} \mathbf{V}_d^{*(k)} - \mathbf{C} - \mathbf{D}^{(k)} \right)$
 - 7: Berechne Leistungsmismatch: $\mathbf{S}_d^{*(k+1)} = \text{diag}(\mathbf{V}_d^{*(k+1)})(Y_{ds} \mathbf{V}_s + Y_{dd} \mathbf{V}_d^{(k+1)})$
 - 8: $\text{tol} \leftarrow \max \|\mathbf{S}_d^* - \mathbf{S}_d^{*(k+1)}\|$
 - 9: $\mathbf{V}_d^0 \leftarrow \mathbf{V}_d^{(k+1)}$ {Arbeitspunkt aktualisieren}
 - 10: $k \leftarrow k + 1$
 - 11: **end while**
 - 12: Berechne $\mathbf{S}_s^* = \text{diag}(\mathbf{V}_s^*)(\mathbf{Y}_{ss} \mathbf{V}_s + \mathbf{Y}_{sd} \mathbf{V}_d^{(k)})$
 - 13: **return** $\mathbf{V}_d^{(k)}$, $\mathbf{S}_s$
-

Die zentrale rechentechnische Eigenschaft des SAM-Algorithmus ist, dass die Matrix $\mathbf{B}^{-1}$ nur *einmal* berechnet werden muss und in allen Iterationen wiederverwendet wird. In jeder Iteration wird lediglich eine Matrix-Vektor-Multiplikation ($\mathbf{B}^{-1} \cdot (\dots)$) durchgeführt [2]. Dies steht im Gegensatz zum Newton-Raphson-Verfahren und zur Direct Solution (DS) aus [2], bei denen in jeder Iteration ein vollständiges lineares Gleichungssystem gelöst werden muss.

Für große Netze kann die direkte Berechnung von $\mathbf{B}^{-1}$ speicherintensiv sein, da die Inverse einer dünnbesetzten Matrix im Allgemeinen vollbesetzt ist. Giraldo et al. [2]

verweisen auf effiziente Algorithmen zum Aufbau der Z-Bus-Matrix [Davis2006] sowie auf die Möglichkeit, $\mathbf{B}$ stattdessen zu faktorisieren (z. B. LDU-Zerlegung) und die Faktorisierung wiederzuverwenden.

1.4.4 Konvergenzbeweis mittels Banach-Fixpunktsatz

Die Konvergenz des SAM-Algorithmus wird von Giraldo et al. [2] über den Banach'schen Fixpunktsatz nachgewiesen. Dieser Satz liefert nicht nur die Existenz und Eindeutigkeit des Fixpunktes, sondern auch die Unabhängigkeit vom Startwert.

Satz 1.2 (Banach'scher Fixpunktsatz). *Sei (X, d) ein vollständiger metrischer Raum und $T : X \rightarrow X$ eine Kontraktion, d. h. es existiert ein $\eta \in [0, 1)$ mit*

$$d(T(\mathbf{x}), T(\mathbf{y})) \leq \eta \cdot d(\mathbf{x}, \mathbf{y}) \quad \forall \mathbf{x}, \mathbf{y} \in X. \quad (1.76)$$

Dann besitzt T genau einen Fixpunkt $\mathbf{x}^ = T(\mathbf{x}^*)$, und die Folge $\mathbf{x}^{(k+1)} = T(\mathbf{x}^{(k)})$ konvergiert für jeden Startwert $\mathbf{x}^{(0)} \in X$ gegen $\mathbf{x}^*$.*

Proposition 1.3 (Kontraktion der SAM, [2, Proposition 2]). *Der SAM-Algorithmus (Algorithmus 1) ist eine Kontraktionsabbildung der Form $\mathbf{V}_d^{(k+1)} = f(\mathbf{V}_d^{(k)})$ und besitzt eine eindeutige, vom Startwert unabhängige Lösung, falls:*

$$\|f(\mathbf{V}_d^{(k)}) - f(\mathbf{U}_d)\| \leq \eta \cdot \|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| \quad (1.77)$$

mit dem Kontraktionsfaktor:

$$\eta = \max_{i \in \Omega_d} \left\{ \frac{\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\|}{v^2} \right\}, \quad 0 \leq \eta < 1 \quad (1.78)$$

wobei $\mathbf{U}_d$ die exakte Lösung ist und $v = \min_{i \in \Omega_d} \|u_i\|$ die minimale Spannungsmagnitude im Netz.

Beweis. Basierend auf dem Banach-Fixpunktsatz existiert der Fixpunkt $\mathbf{U}_d$ mit $f(\mathbf{U}_d) = \mathbf{U}_d$ und ist eindeutig, wenn $f(\cdot)$ eine Kontraktion auf $\mathbf{V}_d$ ist. Die Differenz zweier aufeinanderfolgender Iterationen ergibt sich nach [2] zu:

$$\|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| = \|f(\mathbf{V}_d^{(k)}) - f(\mathbf{U}_d)\| = \|\mathbf{B}^{-1} \mathbf{A}^{(k)} (\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*)\| \quad (1.79)$$

Da $\mathbf{A}^{(k)}$ eine Diagonalmatrix ist, kann abgeschätzt werden:

$$\|\mathbf{V}_d^{(k+1)} - \mathbf{U}_d\| \leq \frac{\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\|}{\|\mathbf{V}_d^{*(k)}\|^2} \cdot \|\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*\| \leq \eta \cdot \|\mathbf{V}_d^{*(k)} - \mathbf{U}_d^*\| \quad (1.80)$$

Damit $\eta < 1$ gilt, müssen zwei Bedingungen erfüllt sein [2]:

1. $v > 0$ (Spannungsbeträge sind im Normalbetrieb stets positiv)
2. $\|\mathbf{B}^{-1} \text{diag}(\alpha_P \odot \mathbf{S}_n^*)\| < v^2$

□

1.4.5 Physikalische Interpretation der Konvergenzbedingung

Die abstrakte Bedingung $\eta < 1$ kann physikalisch interpretiert werden. Unter der Annahme rein leistungskonstanter Last ($\alpha_P = 1$) wird die Matrix $\mathbf{B} = \mathbf{Y}_{dd}$ und somit $\mathbf{B}^{-1} = \mathbf{Z}_B$ die Bus-Impedanzmatrix (Annahme 2 in [2]). Definiert man die äquivalente Lastimpedanz am Knoten i als:

$$Z_{L,i} = \frac{v^2}{\|S_{n,i}^*\|} \quad (1.81)$$

und nutzt die Eigenschaft, dass die Diagonalelemente der Bus-Impedanzmatrix Z_{ii} die Thévenin-Impedanzen darstellen (mit $\|Z_{ii}\| \geq \|Z_{ij}\|$ für $i \neq j$), so ergibt sich der Kontraktionsfaktor näherungsweise zu [2]:

$$\eta \approx \max_{i \in \Omega_d} \left\{ \frac{\|Z_{ii}\|}{\|Z_{L,i}\|} \right\} \quad (1.82)$$

Die Bedingung $\eta < 1$ ist somit äquivalent zu:

$$\|Z_{ii}\| < \|Z_{L,i}\| \quad \forall i \in \Omega_d \quad (1.83)$$

Diese Ungleichung besagt: *Die Thévenin-Impedanz an jedem Knoten muss betragsmäßig kleiner sein als die äquivalente Lastimpedanz.* Nach dem Theorem der maximalen Leistungsübertragung [3] sind beide Impedanzen genau am Punkt maximaler Leistungsübertragung (*maximum loading point*, MLP) gleich groß [2]. In diesem Grenzfall gilt $\eta = 1$ und die Konvergenz ist nicht mehr gewährleistet.

Daraus folgen zwei zentrale Aussagen:

1. Die Bedingung (1.83) ist für den *normalen Betriebspunkt* eines Netzes stets erfüllt, da dort $\|Z_{L,i}\| \gg \|Z_{ii}\|$ gilt (hohe Spannung, niedriger Strom). Der SAM-Algorithmus konvergiert somit stets zur operationellen Lösung, unabhängig vom Startwert.
2. Die Konvergenzrate verschlechtert sich mit zunehmender Last ($\eta \rightarrow 1$ für $\lambda \rightarrow \lambda_{\text{mlp}}$) und mit steigendem Impedanzbetrag der Leitungen ($\|Z_{ii}\|$ wächst mit dem R/X -Verhältnis).

Diese physikalische Interpretation wurde auch in [1] aufgegriffen und dort durch eine geometrische Herleitung im Impedanzraum ergänzt (vgl. ??).

Kapitel 2

Der Tensor Power Flow

Aufbauend auf den in Kapitel 1 eingeführten Grundlagen der Fixed-Point-Iteration wird in diesem Kapitel der Tensor Power Flow (TPF) nach Salazar Duque et al. [1] vorgestellt. Der TPF erweitert den eindimensionalen FPI-Algorithmus zu einer mehrdimensionalen Formulierung, die Hunderttausende von Lastflüssen gleichzeitig berechnen kann. Dabei werden zwei Formulierungen präsentiert sowie deren Eignung für CPU- und GPU-Parallelisierung diskutiert. Das Kapitel schließt mit einer Analyse der fundamentalen Limitation des TPF: der fehlenden Unterstützung von PV-Knoten.

2.1 Grundalgorithmus

Der TPF basiert auf dem in Abschnitt 1.4 eingeführten FPI-Algorithmus nach Giraldo et al. [2]. Ausgangspunkt ist das Netzmodell aus (1.1) mit einem Einspeiseknoten, $b = |\Omega_d|$ Lastknoten und $\phi = |\Omega_\phi|$ Phasen. Die Iterationsvorschrift lautet [1]:

$$\mathbf{v}_{(n+1)} = \mathbf{F} \cdot \mathbf{v}_{(n)}^{*\circ(-1)} + \mathbf{w} \quad (2.1)$$

wobei $\mathbf{v}_{(n)}^{*\circ(-1)} \in \mathbb{C}^{b\phi \times 1}$ den element-weisen Kehrwert des konjugierten Spannungsvektors der n -ten Iteration bezeichnet. Die Operation $(\cdot)^{*\circ(-1)}$ ist definiert als:

$$\left(\mathbf{v}^{*\circ(-1)}\right)_i = \frac{1}{v_i^*} \quad \forall i = 1, \dots, b\phi \quad (2.2)$$

Die konstanten Hilfsmatrizen und -vektoren sind definiert als [1]:

$$\mathbf{A} = \text{diag}(\alpha_P \odot \mathbf{s}^*) \quad (2.3)$$

$$\mathbf{B} = \text{diag}(\alpha_Z \odot \mathbf{s}^*) + Y_{dd} \quad (2.4)$$

$$\mathbf{c} = Y_{ds}\mathbf{v}_s + \alpha_I \odot \mathbf{s}^* \quad (2.5)$$

$$\mathbf{F} = -\mathbf{B}^{-1}\mathbf{A} \quad (2.6)$$

$$\mathbf{w} = -\mathbf{B}^{-1}\mathbf{c} \quad (2.7)$$

Dabei bezeichnen α_P , α_I und α_Z die Koeffizienten des ZIP-Lastmodells (vgl. Abschnitt 1.1.4), $\mathbf{s} \in \mathbb{C}^{b\phi \times 1}$ den Vektor der komplexen Knotenleistungen und $\odot$ das Hadamard-Produkt. Die Matrizen $\mathbf{A} \in \mathbb{C}^{b\phi \times b\phi}$, $\mathbf{B} \in \mathbb{C}^{b\phi \times b\phi}$ und der Vektor $\mathbf{c} \in \mathbb{C}^{b\phi \times 1}$ sind für einen gegebenen Betriebspunkt *konstant* innerhalb des iterativen Prozesses. Folglich können $\mathbf{F} \in \mathbb{C}^{b\phi \times b\phi}$ und $\mathbf{w} \in \mathbb{C}^{b\phi \times 1}$ einmal vorausberechnet und in jeder Iteration wiederverwendet werden.

Für den in Verteilnetzen häufigen Spezialfall rein konstanter Leistung ($\alpha_P = 1$, $\alpha_I = \alpha_Z = 0$) vereinfachen sich die Ausdrücke zu:

$$\mathbf{B} = Y_{dd} \quad (2.8)$$

$$\mathbf{F} = -Y_{dd}^{-1} \text{diag}(\mathbf{s}^*) = -Z_B \text{diag}(\mathbf{s}^*) \quad (2.9)$$

$$\mathbf{w} = -Z_B Y_{ds}\mathbf{v}_s \quad (2.10)$$

wobei $Z_B = Y_{dd}^{-1}$ die Bus-Impedanzmatrix bezeichnet. Die Iteration (2.1) nimmt dann die kompakte Form an:

$$\mathbf{v}_{(n+1)} = Z_B \cdot \left(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*\circ(-1)} \right) + \mathbf{w} \quad (2.11)$$

Der Term $\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*\circ(-1)} = \mathbf{s}^* \oslash \mathbf{v}_{(n)}^*$ (mit $\oslash$ als element-weiser Division) entspricht physikalisch dem konjugierten Stromvektor $\mathbf{i}^* = \mathbf{s}^*/\mathbf{v}^*$, wobei die Beziehung $s_k = v_k \cdot i_k^*$ aus (1.4) genutzt wurde. Die Multiplikation $Z_B \cdot \mathbf{i}^*$ liefert den Spannungsabfall über die Netzipedanzen, und $\mathbf{w}$ korrigiert für den Einfluss der festen Slack-Spannung $\mathbf{v}_s$.

2.2 DENSE- UND SPARSE-FORMULIERUNG

Die zentrale Innovation des TPF besteht in der Erweiterung des eindimensionalen FPI-Algorithmus (2.1) zu einer mehrdimensionalen (tensorisierten) Formulierung, die eine große Anzahl τ von Lastflüssen gleichzeitig verarbeiten kann.

2.2.1 Motivation: Der Tensor der Lastdaten

In typischen Anwendungen wie Zeitreihensimulationen, probabilistischen Lastflüssen oder maschinellem Lernen liegen die Lastdaten als mehrdimensionaler Tensor vor:

$$\mathcal{S} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi} \quad (2.12)$$

wobei die Dimensionen beispielsweise Experimente (p), Szenarien (r) und Zeitschritte (t) repräsentieren können. Um die Berechnung effizient zu gestalten, werden alle nicht-physikalischen Dimensionen zu einem einzigen Index $\tau = \dots \times p \times r \times t$ zusammengefasst (*Reshape-Operation*). Die resultierenden zweidimensionalen Matrizen werden mit der Punkt-Notation gekennzeichnet:

$$\mathcal{S} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi} \xrightarrow{\text{reshape}} \dot{S} \in \mathbb{C}^{b\phi \times \tau} \quad (2.13)$$

Jede Spalte von $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$ enthält den Leistungsvektor eines einzelnen Lastflusses, und die τ Spalten repräsentieren alle zu berechnenden Szenarien. Analog werden die Spannungsmatrizen $\dot{V}_{(n+1)}, \dot{V}_{(n)}^{*o(-1)} \in \mathbb{C}^{b\phi \times \tau}$ definiert.

2.2.2 Dense-Formulierung

Die tensorisierte FPI in ihrer dense Formulierung lautet [1]:

$$\mathcal{V}_{(n+1)} = \mathcal{F} \cdot \mathcal{V}_{(n)}^{*o(-1)} + \mathcal{W} \quad (2.14)$$

mit den Tensor-Dimensionen $\mathcal{F} \in \mathbb{C}^{\dots \times p \times r \times t \times b\phi \times b\phi}$ und $\mathcal{V}_{(n+1)}, \mathcal{V}_{(n)}^{*o(-1)}, \mathcal{W} \in \mathbb{C}^{\dots \times p \times r \times b\phi \times t}$. Im Spezialfall konstanter Leistung und fester Netztopologie besteht der Tensor $\mathcal{F}$ lediglich aus Wiederholungen der identischen Matrix Z_B . Ebenso ist $\mathcal{W}$ eine Wiederholung des konstanten Vektors $\mathbf{w}$. Durch den Reshape zu zweidimensionalen Matrizen ergibt sich die effiziente Berechnungsvorschrift aus Algorithm 1 in [1]:

$$\dot{V}_{(n+1)}[:, i] = Z_B \left(\dot{S}^*[:, i] \odot \dot{V}_{(n)}^{*o(-1)}[:, i] \right)^* + \mathbf{w} \quad \forall i = 1, \dots, \tau \quad (2.15)$$

wobei $[:, i]$ die i -te Spalte der jeweiligen Matrix bezeichnet. Diese τ Operationen können als eine einzige Matrix-Matrix-Multiplikation zusammengefasst werden:

$$\dot{V}_{(n+1)} = Z_B \cdot \underbrace{\left(\dot{S}^* \odot \dot{V}_{(n)}^{*o(-1)} \right)}_{\in \mathbb{C}^{b\phi \times \tau}} + \dot{W} \quad (2.16)$$

wobei $\dot{W} = [\mathbf{w}, \mathbf{w}, \dots, \mathbf{w}] \in \mathbb{C}^{b\phi \times \tau}$ den τ -fach duplizierten Konstantvektor enthält und die Operationen $\odot$ und $(\cdot)^{*o(-1)}$ element-weise auf die gesamten Matrizen angewendet werden.

Gleichung (2.16) stellt eine Multiplikation einer dichten Matrix $Z_B \in \mathbb{C}^{b\phi \times b\phi}$ mit einer Matrix $\in \mathbb{C}^{b\phi \times \tau}$ dar. Dies ist die Schlüsseloperation des TPF: Alle τ Lastflüsse werden durch *eine einzige* Matrix-Matrix-Multiplikation gleichzeitig berechnet.

Algorithm 2 Tensor Power Flow – Dense-Formulierung nach [1]

Require: $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$, $Z_B \in \mathbb{C}^{b\phi \times b\phi}$, $\mathbf{w} \in \mathbb{C}^{b\phi \times 1}$, Toleranz ε , max. Iterationen K

Ensure: $\dot{V}_n \in \mathbb{C}^{b\phi \times \tau}$, Iterationszahl n

```

1:  $\dot{V}_0 = \mathbf{1} + \mathbf{j}\mathbf{0} \in \mathbb{C}^{b\phi \times \tau}$ ;  $n = 0$ ;  $\text{tol} = \infty$ 
2: while  $\text{tol} \geq \varepsilon$  und  $n < K$  do
3:   for  $i = 1$  bis  $\tau$  do
4:      $\dot{V}_{(n+1)}[:, i] = Z_B \left( \dot{S}[:, i] \odot \dot{V}_{(n)}^{*o(-1)}[:, i] \right)^* + \mathbf{w}$ 
5:   end for
6:    $\text{tol} = \max \left( \|s_{(n+1)} - s_{(n)}\|_2 \right)$ ;  $n = n + 1$ 
7: end while
8: return  $\dot{V}_{(n)}$ ,  $n$ 
```

Es ist anzumerken, dass der Reshape von (2.14) zu (2.16) die Konvergenz des FPI-Algorithmus nicht beeinflusst, da die Aktualisierung der Spannungswerte identisch zu (2.1) bleibt – lediglich τ Instanzen werden parallel ausgeführt [1].

2.2.3 Sparse-Formulierung

Ein Nachteil der dense Formulierung ist, dass die Matrix $Z_B = Y_{dd}^{-1}$ vollbesetzt (dense) ist, obwohl die Admittanzmatrix Y_{dd} dünnbesetzt (sparse) ist. Für große Netze ($b\phi > 1000$) kann die Speicherung von Z_B problematisch werden. Salazar Duque et al. [1] schlagen daher eine sparse Formulierung vor, die die Inversion von Y_{dd} vermeidet.

Die Iteration (2.1) wird umformuliert zu:

$$\mathcal{M} \cdot \mathcal{V}_{(n+1)} = \mathcal{V}_{(n)}^{*o(-1)} + \mathcal{H} \quad (2.17)$$

wobei:

$$\mathcal{M} = -\mathbf{A}^{o(-1)} \mathbf{B} \quad (2.18)$$

$$\mathcal{H} = \mathbf{A}^{o(-1)} \mathbf{C} \quad (2.19)$$

mit $\mathbf{A}$, $\mathbf{B}$ und $\mathbf{C}$ gemäß (2.3)–(2.5). Da $\mathbf{A}$ eine Diagonalmatrix ist, überträgt sich die Dünnbesetztheit von Y_{dd} auf $\mathcal{M}$.

Die Reshaped-Form von (2.17) ergibt ein lineares Gleichungssystem der Struktur $\mathbf{A}\mathbf{x} = \mathbf{b}$:

$$\underbrace{\dot{M}}_{\mathbf{A}} \cdot \underbrace{\dot{V}_{(n+1)}}_{\mathbf{x}} = \underbrace{\dot{V}_{(n)}^{*o(-1)} + \dot{H}}_{\mathbf{b}} \quad (2.20)$$

Die Matrix $\dot{M} \in \mathbb{C}^{(\tau \cdot b\phi) \times (\tau \cdot b\phi)}$ ist die Block-Diagonalverkettung der Submatrizen:

$$\dot{M} = \begin{bmatrix} \mathbf{M}_1 & 0 & \cdots & 0 \\ 0 & \mathbf{M}_2 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{M}_\tau \end{bmatrix} \quad (2.21)$$

Die Vektoren $\dot{V}_{(n+1)}$, $\dot{V}_{(n)}^{*\circ(-1)}$ und $\dot{H}$ werden vertikal angeordnet. Das System (2.20) wird iterativ mittels eines sparse Direktl6sers berechnet.

Algorithm 3 Tensor Power Flow – Sparse-Formulierung nach [1]

Require: $\dot{S} \in \mathbb{C}^{b\phi \times \tau}$, $Y_{dd} \in \mathbb{C}^{b\phi \times b\phi}$ (sparse), $Y_{ds} \in \mathbb{C}^{b\phi \times \phi}$ (sparse), Toleranz ε , max. Iterationen K

Ensure: $\dot{V}_n \in \mathbb{C}^{b\phi \times \tau}$, Iterationszahl n

- 1: Berechne $\dot{M}$ und $\dot{H}$ (Block-Diagonal-Aufbau gem66 (2.21))
 - 2: $\dot{V}_0 = \mathbf{1} + \mathbf{j0} \in \mathbb{C}^{(\tau \cdot b\phi) \times 1}$; $n = 0$; $\text{tol} = \infty$
 - 3: **while** $\text{tol} \geq \varepsilon$ **und** $n < K$ **do**
 - 4: $\dot{V}_{(n+1)} = \text{solve}(\dot{M}, \dot{V}_{(n)}^{*\circ(-1)} + \dot{H})$ {Sparse-L6ser}
 - 5: $\text{tol} = \max(\|s_{(n+1)} - s_{(n)}\|_2)$; $n = n + 1$
 - 6: **end while**
 - 7: $\mathbf{V}_n = \text{reshape}(\dot{V}_n, b\phi \times \tau)$
 - 8: **return** $\mathbf{V}_n, n$
-

2.3 Limitation: Keine PV-Knoten

Die Effizienz des Tensor Power Flow basiert auf einer fundamentalen Annahme: *Der Leistungsvektor $\mathbf{s}$ ist f6ur alle τ Lastfl6sse vollst6ndig bekannt und bleibt w6ahrend der gesamten Iteration konstant.*

Diese Annahme erm6glicht:

1. Die **Vorausberechnung** der Systemmatrix Z_B (bzw. $\dot{M}$) und des Konstantvektors $\mathbf{w}$.
2. Die **Wiederverwendung** dieser Gr66en 6uber alle Iterationen und alle τ Lastfl6sse.
3. Die **Zusammenfassung** aller τ Lastfl6sse in eine einzige Matrix-Matrix-Multiplikation.

Ohne diese Konstanzheit w6are der Tensor-Reshape (2.13) nicht durchf6uhrbar, da jeder Lastfluss seine eigene, sich 6andernde Systemmatrix ben6otigen w6urde.

2.3.1 Warum PV-Knoten diese Annahme verletzen

Bei PV-Knoten ist die Blindleistung Q_k unbekannt (vgl. Abschnitt 1.1.3). Der Leistungsvektor am PV-Knoten k lautet:

$$s_k = P_k^{\text{spec}} + j \underbrace{Q_k}_{\text{unbekannt}} \quad (2.22)$$

Da Q_k iterativ bestimmt werden muss, um die Spannungsbedingung $|V_k| = |V_k^{\text{spec}}|$ zu erfüllen, ändert sich s_k zwischen den Iterationen. Dies hat folgende Konsequenzen für die TPF-Formulierung:

1. Die Matrix $\mathbf{A} = \text{diag}(\alpha_P \odot \mathbf{s}^*)$ aus (2.3) ist **nicht mehr konstant**, da $\mathbf{s}$ sich ändert.
2. Damit sind auch $\mathbf{F} = -\mathbf{B}^{-1}\mathbf{A}$ und $\mathbf{w} = -\mathbf{B}^{-1}\mathbf{c}$ (Letzteres über $\mathbf{c}$) nicht mehr über die Iterationen konstant.
3. Im Fall rein konstanter Leistung ($\alpha_P = 1$): Die Iteration $\mathbf{v}_{(n+1)} = Z_B(\mathbf{s}^* \odot \mathbf{v}_{(n)}^{*o(-1)} + \mathbf{w})$ ändert sich nicht in der Systemmatrix Z_B selbst, aber der Leistungsvektor $\dot{S}$ im Produkt $\dot{S}^* \odot \dot{V}_{(n)}^{*o(-1)}$ ist für PV-Knoten iterationsabhängig.
4. Im Tensor-Setting: Verschiedene Szenarien i und j haben im Allgemeinen unterschiedliche Q -Werte am selben PV-Knoten. Die Matrix $\dot{S}$ ändert sich daher sowohl über die Iterationen als auch – nach jeder Q -Korrektur – szenarioabhängig.

Die Limitation des TPF auf PQ-Knoten ist in der Praxis für Verteilnetze oft akzeptabel, da dort die Mehrzahl der Einspeiser (PV-Anlagen mit Wechselrichter, Batteriespeicher) als PQ-Knoten mit festem Leistungsfaktor modelliert wird [1]. Mit zunehmender Integration von Synchrongeneratoren und spannungsregelnden Einheiten in Verteilnetzen gewinnt die korrekte Abbildung von PV-Knoten jedoch an Bedeutung. Die Entwicklung geeigneter Erweiterungen des TPF zur Berücksichtigung von PV-Knoten ist Gegenstand von ??.

Todo list

Literaturverzeichnis

- [1] E. M. Salazar Duque, J. S. Giraldo, P. P. Vergara, P. H. Nguyen und H. Sloomweg, “Tensor Power Flow Formulations for Multidimensional Analyses in Distribution Systems,” *arXiv preprint arXiv:2403.04578v1*, 2024, [eess.SY].
- [2] J. S. Giraldo, O. D. Montoya, P. P. Vergara und F. Milano, “A fixed-point current injection power flow for electric distribution systems using Laurent series,” *Electric Power Systems Research*, Jg. 211, S. 108–126, 2022. DOI: [10.1016/j.epsr.2022.108326](https://doi.org/10.1016/j.epsr.2022.108326)
- [3] J. J. Grainger und W. D. Stevenson, *Power Systems Analysis*. New York: McGraw-Hill, 1994.
- [4] V. M. da Costa, N. Martins und J. L. R. Pereira, “Developments in the Newton Raphson Power Flow Formulation Based on Current Injections,” *IEEE Transactions on Power Systems*, Jg. 14, Nr. 4, S. 1320–1326, Nov. 1999. DOI: [10.1109/59.801890](https://doi.org/10.1109/59.801890)
- [5] P. A. N. Garcia, J. L. R. Pereira, S. Carneiro, V. M. da Costa und N. Martins, “Three-Phase Power Flow Calculations Using the Current Injection Method,” *IEEE Transactions on Power Systems*, Jg. 15, Nr. 2, S. 508–514, 2000. DOI: [10.1109/59.867133](https://doi.org/10.1109/59.867133)

Anhang A

Ergänzende Herleitungen

A.1 Vollständige Herleitung der Sensitivitätskette

A.2 Koeffizienten der quadratischen Gleichung

Anhang B

Zusätzliche Simulationsergebnisse